

# ui.input\_chips 全面详解（基于 NiceGUI 文档）

ui.input\_chips 是 NiceGUI 中用于实现「输入框 + 标签芯片（Chip）」复合交互的组件，基于 Quasar 的 QInput 和 QChip 组件组合实现。其核心价值在于将“文本输入”与“标签管理”深度融合，支持输入文本后快速创建标签、一键删除标签、标签与输入框联动，适用于标签录入、关键词筛选、多值表单填写等需要收集多个离散信息的场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

## 一、核心基础

### 1. 组件本质与核心特性

- 底层依赖：**基于 Quasar 的 QInput（输入框）和 QChip（标签芯片）组件组合，继承输入框的文本输入能力和芯片的标签管理能力。
- 核心逻辑：**输入框用于输入文本，按回车 / 逗号 / 空格可将输入文本转为标签芯片；标签芯片支持一键删除，删除后自动更新组件值；组件值为标签文本组成的列表（`list[str]`），与标签状态实时同步。
- 灵活扩展性：**支持自定义标签颜色、输入分隔符、占位提示，可绑定数据对象实现双向同步，支持禁用状态和样式定制。

### 2. 初始化参数（核心配置）

| 参数名         | 类型              | 说明                                            | 默认值                        |
|-------------|-----------------|-----------------------------------------------|----------------------------|
| value       | list[str]       | 初始标签列表（文本字符串组成的列表）                            | []                         |
| on_change   | Callable        | 标签列表变化时触发的回调函数（ <code>e.value</code> 为当前标签列表） | -                          |
| placeholder | str             | 输入框未输入时的提示文本                                  | ""                         |
| separator   | str / list[str] | 输入分隔符（触发文本转标签的字符），支持单个字符或多个字符列表               | [';', ',', '\n']（逗号、空格、回车） |
| color       | str / None      | 标签芯片的背景色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）     | None（继承 Quasar 默认芯片颜色）     |
| text_color  | str / None      | 标签芯片的文本颜色（优先级高于背景色默认文本色）                      | None                       |
| disabled    | bool            | 是否禁用组件（禁用后无法输入文本、添加 / 删除标签）                   | False                      |

## 二、基础使用示例

### 1. 最简用法（默认配置）

展示默认分隔符（逗号 / 空格 / 回车）、标签创建与删除、状态监听的基础用法：

```

from nicegui import ui

# 最简输入标签组件，监听标签变化
input_chips = ui.input_chips(
    placeholder='输入标签，按回车/逗号/空格分隔...',
    on_change=lambda e: ui.notify(f'当前标签: {e.value}')
)

# 显示当前标签列表（实时同步）
ui.label().bind_text_from(input_chips, 'value', lambda x: f'已选标签: {x}')

ui.run()

```

效果：

- 输入框中输入文本后，按回车、逗号或空格，文本自动转为标签芯片（显示在输入框左侧）；
- 每个标签芯片右侧有“×”按钮，点击可删除该标签；
- 标签添加 / 删除时触发 `on_change` 回调，下方标签实时显示当前标签列表。

## 2. 自定义分隔符与标签颜色

修改标签分隔符（仅支持回车）、自定义标签背景色和文本色，适配特定输入场景：

```

from nicegui import ui

ui.input_chips(
    placeholder='输入标签，按回车分隔...',
    separator=['\n'], # 仅回车触发标签创建
    color='indigo-500', # 标签背景色（Tailwind 颜色）
    text_color='white', # 标签文本色
    value=['初始标签1', '初始标签2'], # 初始标签列表
    on_change=lambda e: ui.notify(f'标签更新: {e.value}')
)

ui.run()

```

效果：

- 仅当输入文本后按回车，才会创建标签（逗号、空格不触发）；
- 标签为靛蓝色背景、白色文本，初始显示两个预设标签；
- 删除或添加标签时触发通知，反馈当前标签列表。

## 三、核心功能与进阶用法

### 1. 数据绑定（双向同步）

通过 `bind_value` 实现组件与数据对象的双向绑定，数据变化时组件自动更新，反之亦然：

```

from nicegui import ui

```

```

class AppState:
    def __init__(self):
        self.tags = ['Python', 'NiceGUI'] # 初始标签数据

state = AppState()

# 组件与数据对象绑定
input_chips = ui.input_chips().bind_value(state, 'tags')

# 显示绑定数据的实时状态
ui.label().bind_text_from(
    state, 'tags',
    lambda x: f'数据对象中的标签: {x}'
)

# 手动修改数据对象，组件自动同步
def add_tag():
    state.tags.append('新增标签')

def clear_tags():
    state.tags.clear()

with ui.row().classes('mt-4 gap-2'):
    ui.button('添加标签', on_click=add_tag)
    ui.button('清空标签', on_click=clear_tags)

ui.run()

```

效果：

- 组件中添加 / 删除标签，`state.tags` 自动同步更新；
- 点击“添加标签”按钮，数据对象新增标签，组件自动显示该标签；
- 点击“清空标签”按钮，数据对象标签列表清空，组件中所有标签自动删除；
- 下方标签实时显示数据对象的标签状态，实现双向绑定。

## 2. 禁用组件与标签操作限制

通过 `disabled` 参数禁用组件，或通过事件拦截限制标签添加 / 删除，适用于权限控制场景：

```

from nicegui import ui

# 初始启用的输入标签组件
input_chips = ui.input_chips(
    placeholder='输入标签（可禁用）...',
    value=['可删除标签'],
    on_change=lambda e: ui.notify(f'标签变化: {e.value}')
)

# 禁用/启用组件的开关
ui.switch('禁用组件', on_change=lambda e: input_chips.set_disabled(e.value)).classes('mt-2')

```

```
# 限制标签最大数量（最多3个）
def limit_tag_count(e):
    if len(e.value) > 3:
        # 超过3个标签时，保留前3个
        input_chips.set_value(e.value[:3])
        ui.notify('最多只能添加3个标签!', type='warning')

input_chips.on_change(limit_tag_count)

ui.run()
```

效果：

- 切换“禁用组件”开关后，组件无法输入文本、添加或删除标签；
- 标签数量超过 3 个时，自动截取前 3 个标签并触发警告通知，限制标签数量。

### 3. 样式定制（输入框 + 标签芯片）

通过 `classes`、`props`、`style` 定制输入框和标签芯片的样式，适配界面设计需求：

```
from nicegui import ui

ui.input_chips(
    placeholder='自定义样式标签...',
    value=['样式1', '样式2'],
    color='teal-400',
    text_color='white'
).props('rounded outlined dense') \ # 输入框：圆角、轮廓样式、紧凑模式
.classes('w-96') \ # 输入框宽度
.style('--q-chip-padding: 4px 8px; --q-chip-border-radius: 16px;') # 标签内边距、圆角

ui.run()
```

效果：

- 输入框为圆角、轮廓样式、紧凑模式，宽度固定为 96 像素；
- 标签芯片为青绿色背景、白色文本，内边距缩小，圆角增大（更圆润）。

### 4. 自定义标签创建逻辑（拦截与校验）

通过 `on_change` 事件拦截标签创建，实现标签校验（如去重、长度限制），优化标签录入体验：

```
from nicegui import ui

def validate_tags(e):
    new_tags = e.value
    validated = []
    for tag in new_tags:
        # 1. 去重：跳过重复标签
        if tag in validated:
            continue
        # 2. 长度限制：标签长度1-10个字符
```

```
        if 1 <= len(tag.strip()) <= 10:
            validated.append(tag.strip()) # 去除首尾空格
        else:
            ui.notify(f'标签「{tag}」无效（长度需1-10个字符）', type='error')
# 更新为校验后的标签列表
input_chips.set_value(validated)

# 输入标签组件，绑定校验逻辑
input_chips = ui.input_chips(
    placeholder='输入标签（去重+长度限制）...',
    separator=['\n'], # 仅回车触发
    on_change=validate_tags
)

ui.run()
```

效果：

- 输入重复标签时，自动去重，仅保留一个；
- 标签长度小于 1 或大于 10 个字符时，触发错误通知，不添加该标签；
- 标签自动去除首尾空格，确保格式统一。

## 四、核心属性与方法

### 1. 常用属性

| 属性名         | 类型               | 说明                                  |
|-------------|------------------|-------------------------------------|
| classes     | Classes[Self]    | CSS 类（支持 Tailwind/Quasar 样式，作用于输入框） |
| disabled    | BindableProperty | 是否禁用组件（可绑定数据动态控制）                   |
| html_id     | str              | HTML 元素 ID（2.16.0+ 版本支持）            |
| placeholder | str              | 输入框提示文本（可动态修改）                      |
| separator   | list[str]        | 标签创建分隔符（可动态修改）                      |
| value       | BindableProperty | 标签列表（list[str]，可绑定数据动态修改）           |
| visible     | BindableProperty | 是否可见（可绑定数据）                         |
| color       | str              | 标签芯片背景色（可动态修改）                      |
| text_color  | str              | 标签芯片文本色（可动态修改）                      |

## 2. 关键方法

### (1) 状态控制

- `disable()`：禁用组件（无法输入、添加 / 删除标签）
- `enable()`：启用组件
- `set_disabled(value: bool)`：设置禁用状态（True/False）
- `set_visibility(visible: bool)`：设置组件可见性

### (2) 属性修改

- `set_value(value: list[str])`：动态设置标签列表（覆盖当前标签）
- `set_placeholder(text: str)`：动态修改输入框提示文本
- `set_separator(separator: str / list[str])`：动态修改标签创建分隔符
- `set_color(color: str)`：动态修改标签芯片背景色
- `set_text_color(text_color: str)`：动态修改标签芯片文本色
- `update()`：触发客户端界面更新（修改属性后需手动调用，绑定数据时无需）

### (3) 事件与绑定

- `on_change(callback)`：绑定标签列表变化事件（`e.value` 为当前标签列表）
- `on(type: str, handler)`：订阅任意 DOM 事件（如 `input`、`click`）
- `bind_value(target_object, target_name)`：双向绑定标签列表到目标对象的属性
- `bind_disabled_from(target_object, target_name)`：单向绑定禁用状态从目标对象

### (4) 其他实用方法

- `tooltip(text: str)`：为组件添加悬停提示
- `delete()`：彻底删除组件
- `mark(*markers)`：添加标记（用于测试或元素查询）
- `focus()`：让输入框获取焦点（激活输入状态）

## 五、使用场景与注意事项

---

### 1. 适用场景

- 标签录入：如文章标签、商品标签、用户兴趣标签等多标签收集场景。
- 关键词筛选：如列表数据的多关键词筛选，输入关键词后转为标签，基于标签筛选结果。
- 多值表单：如表单中的“兴趣爱好”“技能列表”等需要填写多个离散值的字段。
- 动态标签管理：如可添加、删除、去重的标签集合，适用于灵活配置场景。

## 2. 关键注意事项

- 分隔符逻辑：默认分隔符为 `[, ' ', '\n']`（逗号、空格、回车），输入这些字符后，当前输入文本会自动转为标签；若需自定义分隔符，需传入字符或字符列表（如 `[';', '\n']` 仅支持分号和回车）。
- 标签值类型：组件 `value` 始终为 `list[str]`，即使无标签时也为空白列表，无需处理 `None` 情况。
- 样式作用范围：`classes`、`props` 主要作用于输入框，标签芯片样式需通过 `color`、`text_color` 或 `style` 直接定制（如通过 CSS 变量修改芯片样式）。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_disabled` 的 `strict` 参数在 3.0.0+ 版本支持，使用时需确认版本。
- 输入拦截限制：若需阻止特定文本转为标签，需在 `on_change` 事件中拦截并修改 `value`，但需注意避免无限循环（修改 `value` 会再次触发 `on_change`，需通过条件判断控制）。

## 六、进阶示例：带搜索建议的输入标签

结合 `ui.autocomplete` 思路，实现输入时显示标签建议，点击建议快速添加标签，提升录入效率：

```
from nicegui import ui

# 标签建议列表（模拟热门标签）
suggested_tags = ['Python', 'JavaScript', 'NiceGUI', 'Quasar', 'Web', 'UI', 'Frontend']
suggestion_container = None # 建议容器

def show_suggestions(e):
    global suggestion_container
    input_text = e.value.strip()
    if not input_text:
        # 输入为空时隐藏建议
        if suggestion_container:
            suggestion_container.set_visibility(False)
        return

    # 筛选匹配的建议（包含输入文本的标签）
    matched = [tag for tag in suggested_tags if input_text.lower() in tag.lower()]
    if not matched:
        if suggestion_container:
            suggestion_container.set_visibility(False)
        return

    # 创建/更新建议容器
    if not suggestion_container:
        suggestion_container = ui.row().classes('mt-1 gap-2 p-2 bg-white shadow-md rounded-md')
        suggestion_container.parent(input_chips.parent) # 与输入框同层级

    # 清空并重新添加建议标签
    suggestion_container.clear()
    for tag in matched:
        def add_suggested_tag(t=tag):
            # 添加建议标签到组件
            current_tags = input_chips.value
```

```

        if t not in current_tags:
            input_chips.set_value([*current_tags, t])
        # 清空输入框并隐藏建议
        input_chips._props['input-value'] = '' # 清空输入框文本
        suggestion_container.set_visibility(False)

        # 建议标签（点击可添加）
        ui.chip(tag, color='gray-200', text_color='black').on_click(add_suggested_tag)

    suggestion_container.set_visibility(True)

# 输入标签组件，绑定建议显示逻辑
input_chips = ui.input_chips(
    placeholder='输入标签或选择建议...',
    separator=['\n'],
    on_change=lambda e: suggestion_container.set_visibility(False) if suggestion_container
else None
)

# 监听输入框文本变化，显示建议
input_chips.on('input', show_suggestions)

ui.run()

```

效果：

- 输入框中输入文本时，实时显示包含该文本的热门标签建议；
- 点击建议标签可快速添加到组件中，无需手动输入完整文本；
- 添加标签后，输入框文本清空，建议容器隐藏；
- 标签自动去重，避免重复添加相同建议标签。